Randomized Algorithm

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

November 19, 2024

Overview

1. Randomized Algorithm
2. Randomized QuickSort
3. Randomized Selection
4. Randomized Closest Pair
   - Design of Algorithm
   - Data structure for Subsquares
5. Books etc.

Deterministic Algorithm

- On input A, determinstic algorithm must produce a correct solution
- The number of computation steps must be same for different runs of algo with same input
- It may be difficult to formulate an algorithm with good runtime for sufficiently large input

How to get solutions quickly? Randomized Algorithm

Randomized Algorithm (1)

- On input A, randomized algorithm uses random choices to produce the correct solution
- It may go through different steps for different runs of algorithm with same input due to random choices.
- **Advantages:**
  - Randomized algorithms are often simple and easy to implement
  - Usually fast with high probability compared to deterministic algorithms.
  - Produces optimum output with very high probability
  - Malicious adversaries cannot generate any worst case input, that would slow down the randomized algorithm
- **Disadvantages:**
  - There is a finite probability of getting incorrect answer. But the probability can be made arbitrarily small by repeated application of randomness
  - Analysis of runtime and probability of getting correct answer is complex
  - Getting true random number is impossible and results depends on quality of random numbers.

Randomized Algorithm (2)

- Compute expected running time of randomized algorithm, as runtime is independent of any particular input.
- Inputs follow a probability distribution; thus, probabilistic analysis of expected running time

Find an a in input A where $n/2$ elements are a and rest b : Las Vegas Vs. Monte Carlo Algorithm

```
 findingA_LV(array A, n)
begin
    repeat
        Randomly select one element out of n elements.
    until 'a' is found
end

findingA_MC(array A, n, k)
begin
    i := 0
    repeat
        Randomly select one element out of n elements.
        i := i + 1
    until i = k or 'a' is found
end
```


Randomized Algorithm (3)

findingA(LV):

- Algo succeeds with probability 1, though number of iterations may vary.
- Expected number of iterations: $\lim_{n\to\infty} \sum_{i=1}^{n} \frac{i}{2^i} = 2$, ie, $\Theta(1)$

findingA(MC):

- if an a is found algo. succeeds, else it fails.
- After$k$ iterations, the probability of finding an a: $P[find\ a] = 1 - (\frac{1}{2})^k$
- The number of iterations is always less than or equal to k. k being constant the run time (expected and absolute) is $\Theta(1)$.

Randomized Quicksort

- Instead of always using $A[r]$ as pivot, select a randomly chosen element from $A[p..r]$

```
RANDOMIZED-QUICKSORT(A, p, r)
if p < r
    q = RANDOMIZED-PARTITION(A, p, r)
    RANDOMIZED-QUICKSORT(A, p, q − 1)
    RANDOMIZED-QUICKSORT(A, q + 1, r)
```

```
RANDOMIZED-PARTITION(A, p, r)
i = RANDOM(p, r)
exchange A[r] with A[i]
return PARTITION(A, p, r)
```

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```


Randomized Quicksort: Worst case time complexity (1)

- $T(n) = \max_{0\le q\le n-1} (T(q) + T(n-q-1)) + \Theta(n)$
- solve by substitution method: Assume $T(n) \le cn^2$
- substitute the guess, ie,

$$\begin{aligned} T(n) &\le \max_{0\le q\le n-1} (cq^2 + c(n-q-1)^2) + \Theta(n) \\ &= c \max_{0\le q\le n-1} (q^2 + (n-q-1)^2) + \Theta(n) \end{aligned}$$

- $q^2 + (n-q-1)^2$ is maximum when $q = 0$ or $q = n-1$.

$$\begin{aligned} d/dq(q^2 + (n-q-1)^2) &= 2q + 2(n-q-1)(-1) \\ &= 2q - 2n + 2q + 2 \\ &= 4q - 2n + 1 \\ d/dq(4q - 2n + 1) &= 4 > 0 \end{aligned}$$

Randomized Quicksort: Worst case time complexity (2)

- $\max_{0 \le q \le n-1} (q^2 + (n-q-1)^2) \le (n-1)^2 = n^2 - 2n + 1$ (substitute $q = 0$)
- $T(n) \le cn^2 - c(2n-1) + \Theta(n)$
- since c can be taken large to dominate $\Theta(n)$, so $T(n) = O(n^2)$
- when partition is unbalanced, $T(n) = \Omega(n^2)$, Thus, Worst case time complexity is $\Theta(n^2)$

Randomized Quicksort:Expected case time complexity (1)

- *PARTITION* is called n times throughout the quicksort, as during every partition a pivot is chosen and that pivot never appears later.
- Let X is the total number of comparisons in the entire quicksort algorithm (line 4 in Partition)
- Then the running time of QUICKSORT is $O(n + X)$
- Let A has elements $z_1, z_2, \cdots, z_n$, where z_i is the ith smallest element, so, define $Z_{ij} = \{z_i, z_{i+1}, \cdots, z_j$
- Need to know **when the algo. compares two elements z_i and z_j in the array and when it does not**
  - note that each pair of elements are compared only once
  - pivot is compared with all the elements only once, as that pivot does not participate later, once it is placed in correct position
- need probabilistic analysis: Let X_{ij} is indicator random variable

$$X_{ij} = 1, \text{when } z_i \text{ and } z_j \text{ is compared} \quad (1)$$
$$= 0, \text{otherwise} \quad (2)$$

Randomized Quicksort:Expected case time complexity (2)

- since each pair compared only once, total number of comparisons X is

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

- taking expectation,

$$E(X) = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right]$$
$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} E[X_{ij}]$$
$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$$

Randomized Quicksort:Expected case time complexity (3)

- Note 1): once a pivot x is chosen with $z_i < x < z_j$, then z_i and z_j are never compared any later time.
- Note 2): if z_i or z_j is pivot, then z_i or z_j is compared with any other item in Z_{ij}, except themselves
- what is the probability that z_i or z_j from Z_{ij} is the first element chosen as a pivot?
- any element of Z_{ij} is equally likely to be the first one chosen as pivot
- Pr { any elemnet is first one chosen as pivot from Z_{ij} } $= \frac{1}{(j-i+1)}$

$$Pr\{ z_i \text{ is compared to } z_j \} = Pr\{ z_i \text{ or } z_j \text{ is first pivot chosen from } Z_{ij}\}$$
$$= Pr\{ z_i \text{ is first pivot chosen from } Z_{ij}\}$$
$$+ Pr\{ z_j \text{ is first pivot chosen from } Z_{ij}\}$$
$$= \frac{1}{(j-i+1)} + \frac{1}{(j-i+1)}, \quad \text{mutually exclusive}$$
$$= \frac{2}{(j-i+1)}$$

Randomized Quicksort:Expected case time complexity (4)

$$
\begin{aligned}
E(X) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{(j-i+1)}, \text{ harmonic series} \\
&= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{(k+1)}, \; k = j-i \\
&< \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{(k)} \\
&= \sum_{i=1}^{n-1} O(\log n) = O(n \log n)
\end{aligned}
$$

Randomized Selection

Selection Problem: find k^{th} smallest element from $A[1..n]$

```
RANDOMIZED-SELECT(A, p, r, i)
if p == r
    return A[p]
q = RANDOMIZED-PARTITION(A, p, r)
k = q - p + 1
if i == k            // the pivot value is the answer
    return A[q]
elseif i < k
    return RANDOMIZED-SELECT(A, p, q - 1, i)
else return RANDOMIZED-SELECT(A, q + 1, r, i - k)
```


Randomized Selection : Time complexity (1)

- Worst case: $\Theta(n^2)$, if partitions always with the largest remaining element
- Expected running time: $O(n)$
- Run time $T(n)$ on input array $A[p..r]$ is a **random variable**; compute $E[T(n)]$
- Randomized partition is equally likely to return any element as pivot
- Thus, for each k such that $1 \leq k \leq n$, the subarray $A[p..q]$ has k elements $\leq$ pivot with probability $\frac{1}{n}$
- Let $X_k = 1$; {the subarray $A[p..q]$ has exactly k elements }, else $X_k = 0$
- $E[X_k] = P[X_k = 0].0 + P[X_k = 1].1 = \frac{1}{n}.1 = \frac{1}{n}$
- Compute an upper bound on time needed for recursive call on largest possible partition size $A[p..q-1]$ or $A[q+1..r]$

Randomized Selection : Time complexity (2)

- X_k has value 1 for exactly one value of k.

$$
\begin{aligned}
T(n) &\leq \sum_{k=1}^{n} X_k.(T(max(k-1, n-k)) + O(n)) \\
&= \sum_{k=1}^{n} X_k.T(max(k-1, n-k)) + \sum_{k=1}^{n} X_k.O(n) \\
&= \sum_{k=1}^{n} X_k.T(max(k-1, n-k)) + O(n); \; \because X_k = 1 \text{ for one } k
\end{aligned}
$$

Randomized Selection : Time complexity (3)

$$E[T(n)]$$
$$\leq E\left[\sum_{k=1}^{n} X_k.T(max(k-1,n-k)) + O(n)\right]$$
$$= \sum_{k=1}^{n} E[X_k.T(max(k-1,n-k))] + O(n); \text{linearity of Expectation}$$
$$= \sum_{k=1}^{n} E[X_k].E[T(max(k-1,n-k))] + O(n); X_k \text{ and } T(max()) \text{ independent rv}$$
$$= \sum_{k=1}^{n} E[X_k].E[T(max(k-1,n-k))] + O(n); \text{using } E[X_k] = \frac{1}{n}$$

What is $max(k-1,n-k)$**?**

$$max(k-1,n-k) = \begin{cases} k-1 & \text{if} k > \lceil n/2 \rceil \\ n-k & \text{if} k \leq \lceil n/2 \rceil \end{cases} \quad (3)$$

Randomized Selection : Time complexity (3)

Note that each term appears twice in $T(max(k-1,n-k))$, from $T(\lceil n/2 \rceil)$ to $T(n-1$ for $k = 1 \cdots n$, thus,

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + O(n)$$

Use substitution to show $E[T(n)] = O(n)$, assume $E[T(n)] \leq cn$ and $T(n) = 1$ for n less that some constant $E[T(n)]$

$$\leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + an$$
$$= \frac{2c}{n}\left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k\right) + an$$

Randomized Selection : Time complexity (4)

$$= \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(\lfloor n/2 \rfloor - 1)\lfloor n/2 \rfloor}{2}\right) + an$$
$$\leq \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(n/2-2)(n/2-1)}{2}\right) + an$$
$$= \frac{2c}{n}\left(\frac{n^2-n}{2} - \frac{n^2/4 - 3n/2 + 2}{2}\right) + an$$
$$= \frac{c}{n}\left(\frac{3n^2}{4} + \frac{n}{2} - 2\right) + an$$
$$= c\left(\frac{3n}{4} + \frac{1}{2} - \frac{2}{n}\right) + an$$
$$\leq \frac{3cn}{4} + \frac{c}{2} + an$$
$$= cn - (\frac{cn}{4} - \frac{c}{2} - an)$$

- To show for large n last expression is atmost cn, ie, $\frac{cn}{4} - \frac{c}{2} - an \geq 0$
- Add $c/2$ to both sides and factor out n to get $n(c/4 - a) \geq c/2$, ie, $(c/4 - a) > 0$, ie, $c > 4a$
- Thus, $n \geq \frac{c/2}{c/4-a} = \frac{2c}{c-4a}$
- For such n and c, $E[T(n)] = O(n)$

Closest Pair Problem and Randomization

Problem: Given a set of points $P = \{p_1, p_2, \cdots, p_n\}$, where each p_i is (x_i, y_i), find the closest pair.

- It uses randomization and dictionary data structure
- Algorithm runs in $O(n)$ expected time, plus $O(n)$ expected dictionary operations.
- Randomness happens as follows:
  - the way in which the algorithm processes the input points
  - when the dictionary is implemented using hashing
- Let $d(p_i, p_j)$ denotes the Euclidean distance between points p_i and p_j. Thus, find the pairs p_i and p_j such that $d(p_i, p_j)$ is minimized.
- Assume that the points are all in the unit square: $0 \leq x_i, y_i < 1$, $i = 1, 2, \cdots, n$

Designing the algorithm

Overview:

- Consider the points in random order, and maintain a current value δ for closest pair
- When a new point p appears, look **in the vicinity** of p to see if any of the previously considered points are at distance less than δ from p
- **If not**, then closest pair has **not changed**, move to the next point in random order
- **Else**, there is a ponit closer to p, update δ and the closest pair

Stages of Algorithm:

- Assume random order of points are $p_1, p_2, \cdots, p_n$
- In each **stage**, closest pair remains constant
- **First stage** : $\delta = d(p_i, p_j)$
- During a stage points get added as long as there is no change in closest pair
- Stage terminates for a point p_i, when some $p_j, j < i$, $d(p_i, p_j) < \delta$
- Thus $\delta = \min_{j:j<i} d(p_i, p_j)$

Designing the algorithm (1)

Overview:

- Consider the points in random order, and maintain a current value δ for closest pair
- When a new point p appears, look **in the vicinity** of p to see if any of the previously considered points are at distance less than δ from p
- **If not**, then closest pair has **not changed**, move to the next point in random order
- **Else**, there is a ponit closer to p, update δ and the closest pair

Stages of Algorithm:

- Assume random order of points are $p_1, p_2, \cdots, p_n$
- In each **stage**, closest pair remains constant
- **First stage** : $\delta = d(p_i, p_j)$
- During a stage points get added as long as there is no change in closest pair
- Stage terminates for a point p_i, when some $p_j, j < i$, $d(p_i, p_j) < \delta$
- Thus $\delta = \min_{j:j<i} d(p_i, p_j)$

Designing the algorithm (2)

Note:

- number of stages depend on the random order of choosing points
- Minimum 1 stage, if first two points chosen are closest
- Maximum $n-2$ stages, if for every point chosen next, there is a change in closest pair

Testing a proposed distance:

- Subdivide the unit square into subsquares, with side $\delta/2$, ie, $N = 2/\delta$ subsquares on each side, N^2 subsquares
- Subsquare S_{st}, for $0 \le s \le N-1$ and $0 \le t \le N-1$ is

$$S_{st} = \{(x, y) : s(\delta/2) \le x < (s+1)(\delta/2); t(\delta/2) \le y < (t+1)(\delta/2)\}$$

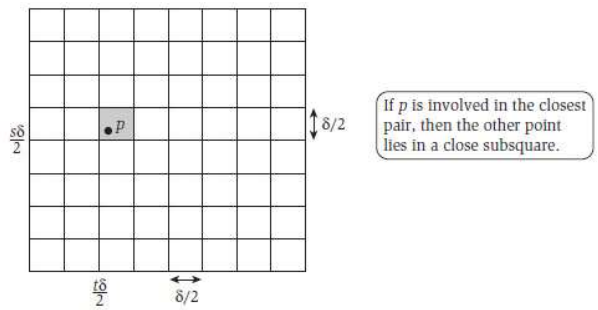
- observe first, that any two points that lie in the same subsquare have distance less than δ.
- Second, any two points that are less than δ away from each other must fall in either the same subsquare or in very close subsquares

Designing the algorithm (3): Lemma 1

Lemma 1: If two points p and q belong to the same subsquare S_{st}, then $d(p, q) < \delta$.

Proof:

- If points p and q are in same subsquare, then both coordinates of the two points differ by at most $\delta/2$, and hence $d(p, q) \le \sqrt{(\delta/2)^2 + (\delta/2)^2}$



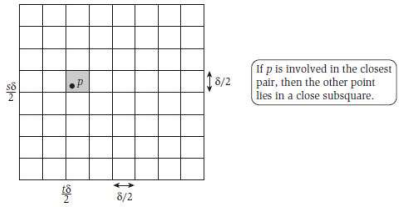
- Subsquares S_{st} and $S_{s't'}$ are close if $|ss'| \le 2$ and $|tt'| \le 2$.

Designing the algorithm (4): Lemma 2

Lemma 2: If for two points $p, q \in P$ we have $d(p, q) < \delta$, then the subsquares containing them are close.

Proof:

- Consider $p, q \in P$ belongs to subsquares that are not close
- Assume $p \in S_{st}$ and $q \in S_{s't'}$, where one of s, s' or t, t' differs by more than 2. ie, in one of their respective x or y-coordinates, p and q differ by at least δ, thus we cannot have $d(p, q) < \delta$.



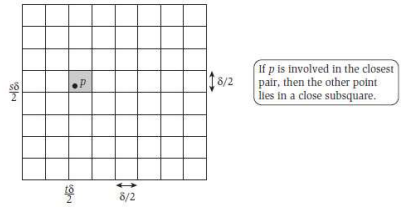
- Note: for any S_{st}, the set of close subsquares is a 5×5 grid around it
- At most 25 subsquares need to be checked for new closest pair for a new point p found in S_{st}

Designing the algorithm (5): Lemma 2

Lemma 2: If for two points $p, q \in P$ we have $d(p, q) < \delta$, then the subsquares containing them are close.

Proof:

- Consider $p, q \in P$ belongs to subsquares that are not close
- Assume $p \in S_{st}$ and $q \in S_{s't'}$, where one of s, s' or t, t' differs by more than 2. ie, in one of their respective x or y-coordinates, p and q differ by at least δ, thus we cannot have $d(p, q) < \delta$.



- Note: for any S_{st}, the set of close subsquares is a 5×5 grid around it
- At most 25 subsquares need to be checked for new closest pair for a new point p found in S_{st}

Designing the algorithm (6): Summary

- Let $P' \subset P$ is set of random points already seen and δ is minimum distance seen so far for P'.
- For each of the points in P', keep track of the subsquare containing it.
- When next point p comes, determine which of thesubsquares S_{st} it belongs to
- If p causes δ to change, there must be some earlier point $p' in P'$ at distance less than δ from it
- Thus, p' must be in one of the 25 squares around S_{st} containing p.
- Check each of these 25 squares to see if it contains a point in P' (Lemma 2)
- To find each point in P' compute its distance to p
- Each of these subsquares contains at most one point of P' (Lemma 1), ie, at most a constant number of distance computations

Randomized Closest Pair Algorithm

```
Order the points in a random sequence p_1, p_2, ..., p_n
Let δ denote the minimum distance found so far
Initialize δ = d(p_1, p_2)
Invoke MakeDictionary for storing subsquares of side length δ/2
For i = 1, 2, ..., n:
  Determine the subsquare S_st containing p_i
  Look up the 25 subsquares close to p_i
  Compute the distance from p_i to any points found in these subsquares
  If there is a point p_j (j < i) such that δ' = d(p_j, p_i) < δ then
    Delete the current dictionary
    Invoke MakeDictionary for storing subsquares of side length δ'/2
    For each of the points p_1, p_2, ..., p_i:
      Determine the subsquare of side length δ'/2 that contains it
      Insert this subsquare into the new dictionary
    Endfor
  Else
    Insert p_i into the current dictionary
  Endif
Endfor
```

References

Thomas H. Cormen,Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms, Third Edition, MIT Press, chapter 32.1,32.3,32.4

The End